석 사 학 위 논 문
Master's Thesis

개체군 기반 자가 대국을 통한 적응형 드론 추적 정책

Adaptive Drone Pursuit Policy via Population-Based Self-Play

2026

마키린 무하마드 카이룰 (基林 穆罕默德 Makirin, Muhammad Khairul)

한 국 과 학 기 술 원
Korea Advanced Institute of Science and Technology

석 사 학 위 논 문

개체군 기반 자가 대국을 통한 적응형 드론 추적 정책

2026

마키린 무하마드 카이룰

한 국 과 학 기 술 원

물리학과

개체군 기반 자가 대국을 통한 적응형 드론 추적 정책

마키린 무하마드 카이룰

위 논문은 한국과학기술원 석사학위논문으로
학위논문 심사위원회의 심사를 통과하였음

2020년 12월 5일

심사위원장 송 익 호 (인)

심 사 위 원 안 진 현 (인)

심 사 위 원 정 태 성 (인)

심 사 위 원 가 동 호 (인)

심 사 위 원 박 태 현 (인)

Adaptive Drone Pursuit Policy via Population-Based Self-Play

Muhammad Khairul Makirin

Advisor: Dong Eui Chang

A dissertation submitted to the faculty of
Korea Advanced Institute of Science and Technology in
partial fulfillment of the requirements for the degree of
Master of Science in Physics

Daejeon, Korea
December 5, 2020

Approved by

Dong Eui Chang
Professor of Electrical Engineering

The study was conducted in accordance with Code of Research Ethics[1].

[1] Declaration of Ethical Conduct in Research: I, as a graduate student of Korea Advanced Institute of Science and Technology, hereby declare that I have not committed any act that may damage the credibility of my research. This includes, but is not limited to, falsification, thesis written by someone else, distortion of research findings, and plagiarism. I confirm that my thesis contains honest conclusions based on my own careful research under the guidance of my advisor.

MPH 마키린무하마드 카이룰. 개체군 기반 자가 대국을 통한 적응형 드론 추적 정책. 물리학과 . 2026년. 21+iv 쪽. 지도교수: 장동의. (영문 논문)
Muhammad Khairul Makirin. Adaptive Drone Pursuit Policy via Population-Based Self-Play. Department of Physics . 2026. 21+iv pages. Advisor: Dong Eui Chang. (Text in English)

초 록

이 논문에서는 부 쓰는이가 여러 안테나를 쓰는 협력 인지 무선통신망을 다루었다. 안테나 개수는 이제까지의 방법과 같도록 유지하면서 부 쓰는이가 다다를 수 있는 전송률을 높이는 방안을 제시하였다. 좀 더 자세히 말하면, 동시 송수신 안테나를 써서 부 쓰는이끼리 전 이중으로 신호를 주고받을 수 있게 하여 부 쓰는이가 다다를 수 있는 전송률을 높이는 방안을 제시하고 그 성능을 살펴보았다. 제안한 협력 인지 무선통신망이 나타내는 다다를 수 있는 전송률이 이제까지의 다른 협력 인지 무선통신망에서 얻을 수 있는 것과 견주어 꽤 높음을 해석적인 방법과 계산적인 방법으로 보였다.

핵 심 낱 말 인지 무선 통신, 협력 통신, 중계, 전 이중, 동시 송수신

Abstract

We address cooperative cognitive radio networks (CCRNs) with secondary users (SUs) exploiting multiple antennas. In order to expand the achievable rate region by enabling full-duplex communication between SUs, we adopt simultaneous transmitting and receiving antennas for the SUs. The link capacities of the proposed framework are analyzed theoretically. It is shown through numerical analysis that the proposed framework can provide a considerable performance gain over the conventional CCRN framework.

Keywords Cognitive radio, cooperation communication, relay, full-duplex, simultaneous transmission and reception

Contents

Contents i
List of Tables iii
List of Figures iv

Chapter 1. Introduction **1**
1.1 Background: Drones as Both Tool and Threat 1
1.2 Learning-Based Pursuit–Evasion and Its Challenges 1
1.3 This Thesis: Adaptive Pursuit via Population-Based Self-Play and an ESN Module 2
1.4 Thesis Organization 3

Chapter 2. Related Works **4**
2.1 Autonomous Drone Interception 4
2.1.1 Classical Guidance and Capture Systems 4
2.1.2 Reinforcement Learning for Agile Flight 4
2.1.3 Reinforcement Learning for Interception 4
2.2 Competitive Multi-Agent Reinforcement Learning and Self-Play 5
2.2.1 Non-Stationarity in Competitive Learning 5
2.2.2 Population-Based Self-Play 5
2.3 Online Adaptation of Learned Policies 5
2.4 Positioning of This Thesis 6

Chapter 3. Proposed Method **7**
3.1 Problem Formulation 7
3.1.1 Pursuit–Evasion as a Markov Game 7
3.1.2 Vehicle and Arena 7
3.1.3 Observation and Action Spaces 7
3.1.4 Episode Termination 8
3.1.5 Reward Design 8
3.2 Population-Based Self-Play 9
3.2.1 Generational Training Loop 9
3.2.2 Multi-Metric Opponent Sampling 9
3.3 Echo State Network Adaptation Module 10
3.3.1 Motivation 10
3.3.2 Reservoir Dynamics 10

3.3.3 Online Readout Update 10
3.3.4 Readout Modes 11

Chapter 4. Experiments, Results, and Discussion **12**
4.1 Experimental Setup 12
4.1.1 Simulation and Training 12
4.1.2 Compared Methods 12
4.1.3 Evader Scenarios 12
4.1.4 Evaluation Metrics 13
4.2 Self-Play Population Analysis 13
4.2.1 Cross-Generation Win-Rate Matrix 13
4.2.2 Effect of the Opponent Sampler 14
4.2.3 Effect of Reward Design 14
4.2.4 Emergent Behaviors 14
4.3 Online Adaptation with the ESN Module 14
4.3.1 Capture Performance on Unseen Evaders 14
4.3.2 Adaptation After a Strategy Switch 14
4.3.3 Ablation of Reservoir Parameters 14
4.3.4 Computational Cost 14
4.4 Discussion 14
4.4.1 Limitations 14

Chapter 5. Conclusion **15**
5.1 Summary 15
5.2 Future Work 15

Bibliography **16**

Acknowledgments in Korean **20**

Curriculum Vitae in Korean **21**

List of Tables

3.1 Terminal rewards 8

4.1 PPO hyperparameters 13
4.2 Capture performance against unseen evaders 14

List of Figures

3.1 Generational self-play algorithm 9

4.1 Cross-generation win-rate matrix 13

Chapter 1. Introduction

1.1 Background: Drones as Both Tool and Threat

Small unmanned aerial vehicles (UAVs) have moved from research laboratories into almost every sector of society, including logistics, agriculture, infrastructure inspection, and public safety. The same properties that make them useful—low cost, high agility, and ease of operation—also make them a growing security concern. Drones have been used for unauthorized surveillance, for disrupting airports and critical infrastructure, and, increasingly, as low-cost weapons in armed conflicts [1]. As a consequence, counter-UAS (C-UAS) capabilities have become a priority for both civilian airspace management and national defense [1, 2].

Among the available countermeasures, physical interception by another drone is attractive because it neutralizes the target without the collateral risks of jamming or kinetic weapons in populated areas [3, 4]. Classical interception systems rely on guidance laws such as proportional navigation together with explicit target-state estimation and prediction [5, 4]. These approaches are well understood and can be deployed on real platforms, but they implicitly assume that the target follows a trajectory that can be extrapolated from its recent motion. A non-cooperative drone that actively reacts to its pursuer violates this assumption: it can dodge, change strategy, or exploit the dynamic limits of the interceptor, turning interception into a genuinely adversarial game.

In parallel, deep reinforcement learning (RL) has transformed what is possible in agile quadrotor flight. Policies trained in simulation now perform acrobatic maneuvers [6], beat human world champions in drone racing [7, 8, 9], and transfer zero-shot to real hardware when trained with low-level collective-thrust-and-body-rate (CTBR) commands [10, 11]. GPU-parallel simulators such as OmniDrones [12] make it feasible to collect billions of interaction steps for such policies. These advances suggest that an interceptor policy can be learned end-to-end, directly from interaction with a moving target, rather than being hand-designed.

1.2 Learning-Based Pursuit–Evasion and Its Challenges

RL has been applied to target interception in a variety of forms, from missile guidance [13] and fixed-wing interception in controlled airspace [14] to vision-based drone interception [15], multi-target tracking [16], and cooperative multi-agent interception [17, 18, 19]. Many of these works, however, train the pursuer against targets with fixed, scripted, or randomly generated behavior. A pursuer trained this way learns to exploit the specific regularities of its training targets and is not guaranteed to remain effective against an evader that behaves strategically.

Modeling the evader as a learning agent turns the problem into a two-player zero-sum Markov game, which falls in the domain of competitive multi-agent reinforcement learning (MARL) [20, 21, 22]. Training both sides simultaneously, however, introduces well-known difficulties: each agent's environment becomes non-stationary because its opponent keeps changing; agents overfit to the current opponent and forget how to handle earlier strategies (catastrophic forgetting); and the dynamics of mutual best responses can lead to strategy cycling rather than steady improvement [23, 24]. Population-based self-play addresses these issues by training against a growing pool of past and diverse opponents instead of only the latest

one. This idea underlies league training in AlphaStar [25], opponent-sampling schemes in autonomous air combat [26], and, more recently, quadrotor pursuit–evasion and racing [27, 24, 28, 29, 30, 31]. In these works, the choice of *which* opponent to train against—uniformly, by prioritized fictitious self-play (PFSP), or by a Hedge-style sampler over win rates—has a decisive effect on the robustness of the final policy [26, 24, 30].

Two gaps remain. First, existing opponent samplers rank opponents almost exclusively by win rate. In physical pursuit–evasion, however, win rate is a coarse signal: an evader can be "hard" because it escapes cleanly, because it drags the capture out over a long time, or because it forces the pursuer into aggressive, crash-prone flight. Conversely, an evader can appear easy simply because it destroys itself by flying into the ground or a wall. A sampler that only sees win rate cannot distinguish these cases and may focus training on the wrong opponents.

Second, even a well-trained population-based policy is *frozen* at deployment. It encodes a response to the mixture of opponents it saw during training, but it has no explicit mechanism to identify and adapt to the particular evader it is currently facing, especially one whose strategy was never seen in training or that changes strategy mid-episode. Opponent modeling [32] and target-prediction networks [18, 19] supply the policy with an estimate of the opponent's future behavior, but they are typically trained offline and not updated online. Rapid online adaptation has been studied extensively for changing *dynamics*, through residual learning, differentiable simulation, and latent-condition inference [33, 34, 35, 36], yet these methods generally rely on gradient-based updates or privileged information, which are costly on resource-constrained flight hardware. Reservoir computing offers a lightweight alternative: an echo state network (ESN) with a fixed random reservoir and a linear readout updated by recursive least squares (RLS) can adapt within a few control steps without backpropagation or pretraining [37]. To the best of our knowledge, such online adaptation has not been combined with population-based self-play for adversarial aerial pursuit.

1.3 This Thesis: Adaptive Pursuit via Population-Based Self-Play and an ESN Module

This thesis develops an adaptive pursuit policy for one-versus-one quadrotor pursuit–evasion that addresses both gaps. The pursuer and the evader are both Crazyflie quadrotors controlled by PPO policies [38] that output CTBR commands tracked by an on-board rate controller, and they are trained in a GPU-parallel Isaac Sim environment built on OmniDrones [12] inside a bounded arena. The approach has two components.

The first component is a generational, population-based self-play framework. Starting from scripted heuristic opponents (hover, circular orbit, pursue, and flee), each generation trains one side against frozen opponents drawn from the other side's population, evaluates the new policy against the entire pool, and adds it back. Instead of sampling opponents by win rate alone, we score each opponent with a temperature-scaled softmax over several normalized *hardness* metrics—capture rate, time to capture, motor effort, and the training agent's own crash and out-of-bounds rate—so that training concentrates on opponents that are difficult in different ways. A reward design with symmetric terminal outcomes, partial credit for opponent-induced failures, and body-rate command penalties prevents degenerate strategies such as waiting for the opponent to crash and holding saturated rate commands that would tumble a real vehicle.

The second component is an ESN-based online adaptation module attached to the pursuer. A fixed random reservoir encodes the pursuer's observation history, and a linear readout predicts the evader's next relative position and velocity in the pursuer's body frame. The readout is updated online by RLS at every control step, and its prediction is appended to the policy input. This gives the pursuer a continually updated, opponent-specific forecast at the cost of a matrix update per step, without gradient computation at test time. We study three readout variants: a shared readout fitted centrally during training and frozen at deployment, a per-episode readout adapted from scratch, and a shared readout used as the initialization for per-episode adaptation.

The main contributions of this thesis are summarized as follows:

- A population-based self-play framework for agile quadrotor pursuit–evasion with CTBR control, including heuristic population seeding, a reward structure that removes degenerate strategies, and a multi-metric, temperature-controlled opponent sampler that extends win-rate-based sampling.
- The integration of an echo state network with online RLS readout into the pursuer policy as a lightweight, gradient-free opponent-prediction and adaptation module, together with shared, per-episode, and hybrid readout schemes.
- An evaluation protocol based on cross-generation win-rate matrices and held-out evader strategies, including strategy switches within an episode, to assess both the robustness of the population and the adaptation ability of the pursuer.

1.4 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 reviews related work on drone interception, competitive multi-agent reinforcement learning and self-play, and online adaptation. Chapter 3 formulates the pursuit–evasion problem and presents the proposed self-play framework and ESN adaptation module. Chapter 4 describes the experimental setup and discusses the results. Chapter 5 concludes the thesis and outlines directions for future work.

Chapter 2. Related Works

This chapter reviews the three research areas on which this thesis builds: autonomous drone interception, competitive multi-agent reinforcement learning with self-play, and online adaptation of learned policies.

2.1 Autonomous Drone Interception

2.1.1 Classical Guidance and Capture Systems

Counter-UAS systems span detection, tracking, and neutralization, with physical capture by an interceptor drone being one of the few options that is safe to use in populated areas [1]. Vrba et al. [4] built a complete LiDAR-based system that predicts the target trajectory and repositions a net-carrying UAV to catch it, achieving top results in the MBZIRC 2020 challenge. Pliska et al. [5] proposed Fast Response Proportional Navigation together with an interacting-multiple-model filter for target estimation, and showed that estimation quality is as important as the guidance law itself. Model-based trajectory generation for fast flight [39] and decision-level allocation of interceptors to swarm threats [2] complement these approaches. A common limitation is that the target is treated as a process to be estimated and extrapolated rather than as an agent that reacts to the interceptor.

2.1.2 Reinforcement Learning for Agile Flight

Deep RL has enabled quadrotor control at the limits of the platform. Policies trained in simulation have performed acrobatic maneuvers [6] and defeated human world champions in drone racing [7], with later systems winning competitions using only monocular vision [8] or operating in uninstrumented arenas [9]. Kaufmann et al. [10] showed that policies commanding collective thrust and body rates (CTBR) transfer to hardware more robustly than those commanding individual rotor thrusts, and Chen et al. [11] identified practical factors—including action smoothness regularization, system identification, and large batch sizes—that enable zero-shot sim-to-real transfer on small quadrotors such as the Crazyflie. GPU-parallel simulators, notably OmniDrones on Isaac Sim [12], have made training at this scale practical. Related lines of work address safe navigation among dynamic obstacles [40, 41], differentiable-physics training [42, 43], and generalization across tasks [36].

2.1.3 Reinforcement Learning for Interception

RL-based interception has been studied for missiles using line-of-sight observations [13], for fixed-wing UAVs in controlled airspace [14], and for multirotors using visual input [15] or obstacle-aware planning in urban scenes [3]. Multi-target and multi-agent variants combine RL with fuzzy logic [16], handle switching communication topologies [17], or coordinate several pursuers with target-prediction networks [18, 19]. Control-theoretic RL formulations of the pursuit–evasion differential game add safety and robustness guarantees [44]. In most of these works the target follows scripted or randomized motion, so the learned pursuer is not exposed to an evader that optimizes against it.

2.2 Competitive Multi-Agent Reinforcement Learning and Self-Play

2.2.1 Non-Stationarity in Competitive Learning

When both sides of a game learn, each agent faces a non-stationary environment, which breaks the convergence assumptions of single-agent RL [20, 21]. Centralized-critic methods such as MADDPG [22] and multi-agent PPO [38] mitigate this in part, while opponent-modeling approaches explicitly predict how the opponent will behave [32]. In purely competitive settings, naive self-play against only the latest opponent is prone to overfitting, forgetting, and cyclic strategies; benchmarks such as FightLadder [23] show that producing non-exploitable agents remains difficult. Feng et al. [45] highlight the same challenges for embodied multi-agent systems more broadly.

2.2.2 Population-Based Self-Play

Population-based training keeps a pool of previous and diverse policies and trains new policies against mixtures of that pool. AlphaStar [25] used a league of main agents and exploiters with prioritized fictitious self-play (PFSP), which samples opponents in proportion to how often they beat the learner. In air combat, Chen et al. [26] proposed automatic opponent sampling with a simulated-annealing-inspired curriculum and Boltzmann meta-solver, and Kong et al. [46] addressed robustness to observation errors.

For quadrotor pursuit–evasion, Xiao and Feroskhan [27] trained pursuers and evader asynchronously over multiple stages, and Roncero et al. [24] extended this to a PSRO-style population with Hedge sampling over win rates, training one side per generation against frozen opponents. AgilePE [28] and Gavin et al. [29, 30] used PFSP with CTBR control and demonstrated zero-shot transfer to real quadrotors, while Geles et al. [31] applied league-based self-play to multi-drone racing. Xu et al. [47] combined self-play PPO with curriculum learning across spacecraft and drone domains. These works establish population-based self-play as an effective tool for aerial pursuit–evasion, but they rank opponents almost exclusively by win rate.

2.3 Online Adaptation of Learned Policies

A policy trained in simulation is typically deployed with frozen weights. To handle conditions not seen in training, recent work adapts the policy online. Pan et al. [33] refine a residual dynamics model from real data and update the policy through differentiable simulation within seconds. Xing et al. [34] infer a latent dynamics condition from recent interaction with a recurrent encoder and condition the policy on it. Pertin and Chang [35] learn residual corrections on top of an approximately optimal regulator. These methods target changes in the robot's own dynamics and rely on gradient updates or learned encoders.

Reservoir computing offers a cheaper route. An echo state network (ESN) uses a fixed, randomly initialized recurrent reservoir whose state encodes the input history; only a linear readout is learned. Yoshimura and Tanaka [37] attached an ESN to an RL policy and updated its readout online by recursive least squares (RLS), achieving adaptation to abrupt changes within a few control steps without backpropagation, pretraining, or privileged information. Their evaluation considered non-stationary dynamics in single-agent benchmarks; applying the same mechanism to a non-stationary *opponent* in an adversarial aerial task has not been explored.

2.4 Positioning of This Thesis

This thesis combines the two threads above. Like [24, 28, 30], it trains CTBR quadrotor policies by population-based self-play, but it replaces win-rate-only opponent sampling with a multi-metric hardness score that also accounts for capture time, control effort, and the learner's own failure rate. Building on [37], it equips the pursuer with an ESN whose RLS readout predicts the evader's relative motion online, providing a gradient-free mechanism for adapting to the specific evader encountered at test time.

Chapter 3. Proposed Method

This chapter formulates the one-versus-one quadrotor pursuit–evasion problem and presents the two components of the proposed approach: a population-based self-play framework with multi-metric opponent sampling (Section 3.2) and an echo state network module for online adaptation of the pursuer (Section 3.3).

3.1 Problem Formulation

3.1.1 Pursuit–Evasion as a Markov Game

We consider a two-player zero-sum Markov game between a pursuer P and an evader E, defined by the tuple $(\mathcal{S}, \mathcal{O}^P, \mathcal{O}^E, \mathcal{A}^P, \mathcal{A}^E, T, r^P, r^E, \gamma)$. At each step t, each agent $i \in \{P, E\}$ receives an observation $o_t^i \in \mathcal{O}^i$ and selects an action $a_t^i \sim \pi^i(\cdot \mid o_t^i)$. The environment transitions according to the quadrotor dynamics T, and each agent receives a reward r_t^i. The pursuer seeks to capture the evader as quickly as possible; the evader seeks to survive until the time limit. When one side is being trained, the other is a fixed policy drawn from a population, so from the learner's point of view the problem reduces to a partially observable MDP whose dynamics depend on the unknown opponent.

3.1.2 Vehicle and Arena

Both agents are Crazyflie quadrotors simulated in Isaac Sim through the OmniDrones framework [12]. The game takes place inside a bounded box of 8 m × 8 m horizontally and 0.1–3.0 m vertically. The two drones are spawned at random positions with a minimum separation of 1 m. To improve robustness, the mass (±5%), thrust-to-weight ratio (±15%), inertia (±10%), and motor time constants (0.08–0.17 s) of the learning agent are randomized at every episode.

3.1.3 Observation and Action Spaces

The observation of each agent is expressed in its own body frame so that a policy trained in one role can be replayed as the opponent of the other role. It consists of

$$o_t^i = \left[\, \mathbf{p}^i,\ \mathrm{vec}(\mathbf{R}^i),\ \mathbf{v}_b^i,\ \boldsymbol{\omega}_b^i,\ \Delta\mathbf{p}_b,\ \Delta\mathbf{v}_b,\ a_{t-1}^i \,\right] \in \mathbb{R}^{28}, \tag{3.1}$$

where $\mathbf{p}^i$ is the agent position, $\mathbf{R}^i$ its rotation matrix, $\mathbf{v}_b^i$ and $\boldsymbol{\omega}_b^i$ its body-frame linear and angular velocities, $\Delta\mathbf{p}_b$ and $\Delta\mathbf{v}_b$ the opponent's position and velocity relative to the agent expressed in the agent's body frame, and a_{t-1}^i the previous action. The critic additionally receives a time encoding of the episode progress.

The action $a_t^i = [\omega_x, \omega_y, \omega_z, T] \in [-1, 1]^4$ specifies desired body rates and normalized collective thrust (CTBR), which an on-board PID rate controller converts to motor commands. CTBR control has been shown to give agile behavior and robust sim-to-real transfer [10, 11, 28].

Table 3.1: Terminal rewards for each outcome. $W = 100$ and $s = 0.25$ in all experiments.

Outcome	Pursuer	Evader
Capture	$+W$	$-W$
Timeout (evader survives)	$-W$	$+W$
Opponent crashes	$+sW$	$+sW$
Own crash	$-W$	$-W$
Evader leaves the arena	0	$-W$

3.1.4 Episode Termination

The pursuer captures the evader when their distance falls below the capture radius $d_c = 0.1\,\mathrm{m}$. An episode terminates on capture, when either drone falls below the minimum altitude (a crash), when the evader leaves the arena, or on a numerically invalid state; it is truncated after $T_{\max} = 1500$ control steps, which counts as a successful escape for the evader.

3.1.5 Reward Design

The reward consists of a terminal term shared by both roles and role-specific dense terms. Let W denote the terminal reward magnitude and $s \in (0, 1)$ a partial-credit factor. The terminal outcomes are summarized in Table 3.1.

The partial credit for an opponent crash is a deliberate design choice. When an opponent crash was rewarded as a full win, the pursuer learned to keep its distance and wait for a scripted evader to fall out of the sky: most of its "wins" came from evader crashes and its true capture rate collapsed. Partial credit keeps a signal for forcing errors while making capture strictly the best outcome.

The pursuer additionally receives an approach reward proportional to the decrease in distance d_t to the evader and a precision potential that pulls it through the final meter:

$$r_t^{P,\mathrm{dense}} = w_a(d_{t-1} - d_t) + w_p\, e^{-k d_t}. \tag{3.2}$$

The evader receives a small per-step survival reward r_s and a penalty that ramps up within a margin of the arena walls, so that it learns the boundary before reaching it.

Both agents are penalized for aggressive control through three complementary terms:

$$r_t^{\mathrm{smooth}} = -w_\omega \|\boldsymbol{\omega}_b\| - w_\Delta \|a_t - a_{t-1}\| - w_c \|\tanh(a_{t,1:3})\|, \tag{3.3}$$

penalizing respectively the actual body rate, the change in action between steps, and the magnitude of the commanded body rate. The third term is essential for deployment: at the training control rate a saturated rate command is never fully realized within one step, so the first term barely penalizes it, and a sustained saturated command has no step-to-step change, so the second term does not either. The real firmware rate loop, however, tracks such commands faithfully and can tumble the vehicle.

Algorithm 1: Generational population-based self-play

Input: heuristic seeds Π_0^P, Π_0^E; number of generations G; temperature τ; metric weights $\{w_m\}$

for $g = 0, \ldots, G-1$ **do**

$i \leftarrow$ pursuer if g is even, else evader; $j \leftarrow$ the other role

compute sampling probabilities $q(\pi^j)$ for all $\pi^j \in \Pi^j$ using Eq. (3.4)

assign each parallel environment an opponent $\pi^j \sim q$

$\pi_g^i \leftarrow \mathrm{PPO}(\mathrm{init} = \pi_{g-1}^i)$ against the assigned opponents

evaluate π_g^i against every $\pi^j \in \Pi^j$ and store matchup metrics

$\Pi^i \leftarrow \Pi^i \cup \{\pi_g^i\}$

end for

Figure 3.1: Generational population-based self-play.

3.2 Population-Based Self-Play

3.2.1 Generational Training Loop

Algorithm 3.1 summarizes the training procedure. Two populations Π^P and Π^E are seeded with scripted heuristics: a pursuing heuristic for the pursuer pool, and hovering, circular-orbit, and fleeing heuristics for the evader pool. Each generation trains one side with PPO [38], alternating between pursuer and evader. The new policy is warm-started from the latest policy of the same role and trained against frozen opponents sampled from the other population. After training, it is evaluated against every opponent in the pool, the resulting matchup statistics are stored, and the policy is added to its population.

Heuristic opponents are flown through the same CTBR and rate-controller stack as the learned policies. Their targets are clamped inside the arena and their tilt is limited, so that they remain physically feasible. The fleeing heuristic in particular escapes horizontally at a fixed altitude and brakes before the walls; a naive three-dimensional flee frequently dived into the ground or crashed into a wall, removing itself from the game before any interception could occur.

3.2.2 Multi-Metric Opponent Sampling

For every matchup between the current reference policy of the learning role and a candidate opponent π^j, the evaluation stage records five metrics: the pursuer capture rate, the mean time to capture, the mean inter-drone distance, the mean motor effort, and the learner's own error rate (fraction of episodes in which the learner crashed or left the arena). Each metric m is min–max normalized across candidates and oriented by a sign $\sigma_m^i \in \{-1, +1\}$ so that a larger value always indicates an opponent that is harder for the learner of role i. For example, a hard evader has a low capture rate and a long capture time, while a hard pursuer has the opposite. The learner's error rate is oriented as harder-when-larger for both roles, since an opponent that forces the learner into risky flight is difficult regardless of which side is learning. The hardness score and sampling distribution are

$$h(\pi^j) = \sum_m w_m \, \sigma_m^i \, \tilde{x}_m(\pi^j), \qquad q(\pi^j) = \frac{\exp\big(h(\pi^j)/\tau\big)}{\sum_k \exp\big(h(\pi^k)/\tau\big)}, \tag{3.4}$$

where $\tilde{x}_m$ is the normalized metric, w_m its weight, and τ a temperature. A low temperature focuses training on the hardest opponents and a high temperature approaches uniform sampling. Without matchup data (cold start), sampling is uniform.

The additional metrics capture aspects of difficulty that win rate alone misses. In particular, the learner's error rate distinguishes an evader that wins by forcing the pursuer to crash from one that wins by clean evasion; both may have similar capture rates but require different counter-strategies. We use $w = (1.0, 0.5, 0.0, 0.25, 1.0)$ for (capture rate, time to capture, distance, motor effort, error rate) and $\tau = 0.9$.

3.3 Echo State Network Adaptation Module

3.3.1 Motivation

A pursuer trained against a population learns a response to the *mixture* of opponents it has seen. At test time, however, it faces one specific evader, possibly with a strategy that was absent from training or that changes during the episode. We therefore provide the pursuer with an online-adapted prediction of the evader's motion, computed by an ESN whose readout is updated by RLS at every step [37]. Because only a linear readout is adapted, the module requires no backpropagation at test time.

3.3.2 Reservoir Dynamics

The ESN input is the pursuer's observation $u_t = o_t^P$. A reservoir of $N_x = 500$ leaky-integrator units evolves as

$$\mathbf{x}_t = (1-\alpha)\,\mathbf{x}_{t-1} + \alpha \tanh\big(\mathbf{W}_{\text{in}}\mathbf{u}_t + \mathbf{W}_{\text{res}}\mathbf{x}_{t-1}\big), \tag{3.5}$$

where $\alpha = 0.3$ is the leak rate. The input weights $\mathbf{W}_{\text{in}}$ are drawn uniformly from $[-0.1, 0.1]$, and the recurrent weights $\mathbf{W}_{\text{res}}$ are drawn uniformly from $[-1, 1]$ and rescaled to a spectral radius of 0.9. Both are fixed after initialization. With the feature vector $\boldsymbol{\phi}_t = [\mathbf{x}_t; 1]$, the readout predicts

$$\hat{\mathbf{y}}_{t+1} = \mathbf{W}_{\text{out}}\,\boldsymbol{\phi}_t, \tag{3.6}$$

where the target $\mathbf{y}_{t+1} = [\Delta\mathbf{p}_b; \Delta\mathbf{v}_b]_{t+1} \in \mathbb{R}^6$ is the evader's relative position and velocity in the pursuer's body frame at the next step. The prediction $\hat{\mathbf{y}}_{t+1}$ is appended to the policy observation.

3.3.3 Online Readout Update

When the true $\mathbf{y}_t$ becomes available, the readout is updated by RLS with forgetting factor $\lambda = 0.99$:

$$\mathbf{e}_t = \mathbf{y}_t - \mathbf{W}_{\text{out}}\boldsymbol{\phi}_{t-1}, \tag{3.7}$$

$$\mathbf{k}_t = \frac{\mathbf{P}_{t-1}\boldsymbol{\phi}_{t-1}}{\lambda + \boldsymbol{\phi}_{t-1}^\top \mathbf{P}_{t-1}\boldsymbol{\phi}_{t-1}}, \tag{3.8}$$

$$\mathbf{W}_{\text{out}} \leftarrow \mathbf{W}_{\text{out}} + \mathbf{e}_t\mathbf{k}_t^\top, \tag{3.9}$$

$$\mathbf{P}_t = \frac{1}{\lambda}\big(\mathbf{P}_{t-1} - \mathbf{k}_t\boldsymbol{\phi}_{t-1}^\top\mathbf{P}_{t-1}\big), \tag{3.10}$$

with $\mathbf{P}_0 = \delta\mathbf{I}$ and $\delta = 100$. To prevent wind-up of $\mathbf{P}$ in weakly excited directions, its trace is capped at 10^6.

3.3.4 Readout Modes

We consider three ways of initializing and updating the readout:

- **Per-episode (ESN-OA).** Each environment starts every episode with $\mathbf{W}_{\text{out}} = \mathbf{0}$ and adapts it with its own RLS, as in [37].
- **Shared.** A single readout is fitted centrally by ridge regression on reservoir features pooled from all parallel environments, using exponentially forgotten normal equations re-solved periodically. It is frozen at deployment, so the module acts as a learned, non-adaptive predictor.
- **Shared-adapt.** Each episode starts from the shared readout and then adapts it with per-environment RLS, combining a good prior with online specialization.

Comparing these modes separates the effect of the extra predictive input from the effect of online adaptation itself.

Chapter 4. Experiments, Results, and Discussion

This chapter evaluates the proposed approach. Section 4.1 describes the experimental setup, Section 4.2 analyzes the self-play population, Section 4.3 evaluates the ESN adaptation module against unseen evaders, and Section 4.4 discusses the findings and limitations.

4.1 Experimental Setup

4.1.1 Simulation and Training

All experiments are run in Isaac Sim with the OmniDrones framework [12] using 4096 parallel environments on a single GPU. Policies are trained with PPO [38]; the hyperparameters are listed in Table 4.1. Self-play runs for 12 generations of 5×10^8 environment frames each, starting with the pursuer and warm-starting every policy from the latest policy of the same role.

4.1.2 Compared Methods

To evaluate the adaptation module, all pursuers are trained with identical PPO settings, rewards, arena, domain randomization, and frame budget; only the adaptation mechanism differs:

- **PPO-MLP**: the baseline policy without any adaptation module.
- **ESN-shared**: ESN with a centrally fitted readout, frozen at test time.
- **ESN-OA**: ESN with a per-episode RLS readout.
- **ESN-shared-adapt**: ESN with the shared readout as initialization and per-episode RLS.
- **PPO-GRU**: a recurrent policy whose hidden state can implicitly infer the evader's behavior.
- **RLS-raw**: the same RLS readout applied to a window of raw observations, without a reservoir.

4.1.3 Evader Scenarios

Pursuers are evaluated on five groups of evaders:

- **T0 (in-distribution):** the heuristics and RL evaders used in training.
- **T1 (new heuristics):** weaving, zig-zag, spiral-climb, and fast-flee evaders not seen during training.
- **T2 (held-out RL evaders):** later-generation self-play evaders excluded from the training pool.
- **T3 (strategy switch):** evaders that switch strategy partway through an episode.
- **T4 (dynamics shift):** evaders whose mass, thrust, or motor time constants lie outside the training randomization range.

Table 4.1: PPO hyperparameters.

Parameter	Value
Parallel environments	4096
Frames per generation	5×10^8
Discount factor γ	0.99
GAE λ	–
Learning rate	–
Clip ratio	–
Actor / critic network	–

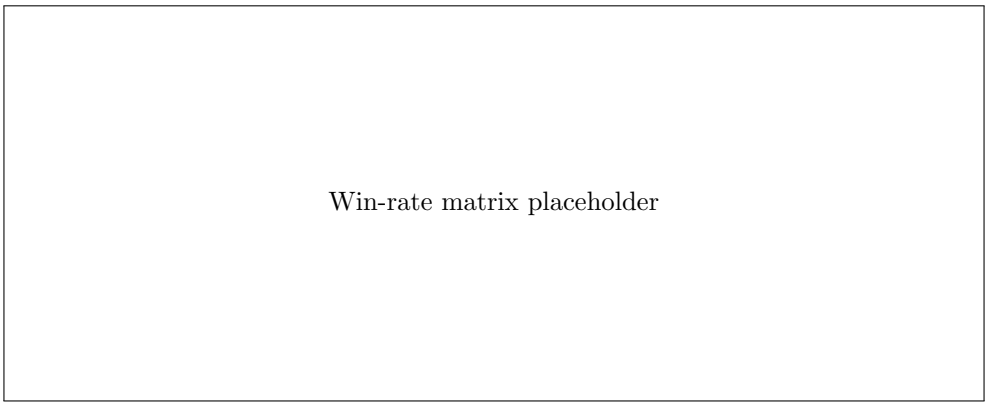


Figure 4.1: Pursuer capture rate for every pursuer generation (rows) against every evader generation (columns). Cells not evaluated are left blank.

4.1.4 Evaluation Metrics

For each method and scenario we report the capture rate, the time to capture, the pursuer's own crash rate, the mean motor effort, and, for methods with a predictor, the prediction error of the evader's relative state. For the strategy-switch scenarios we additionally report the recovery time, defined as the time after the switch until the pursuer's rate of closing distance returns to its pre-switch level. Results are averaged over three training seeds and at least 4096 evaluation episodes per seed, with 95% bootstrap confidence intervals.

4.2 Self-Play Population Analysis

4.2.1 Cross-Generation Win-Rate Matrix

Figure 4.1 shows the capture rate of each pursuer generation against each evader generation. A healthy population exhibits a staircase pattern: each pursuer beats the evaders it was trained against and loses to the newer evaders trained to counter it.

Table 4.2: Capture rate (%) per method and scenario (mean ± 95% CI over seeds).

Method	T0	T1	T2	T3	T4
PPO-MLP	–	–	–	–	–
ESN-shared	–	–	–	–	–
ESN-OA	–	–	–	–	–
ESN-shared-adapt	–	–	–	–	–
PPO-GRU	–	–	–	–	–
RLS-raw	–	–	–	–	–

4.2.2 Effect of the Opponent Sampler

4.2.3 Effect of Reward Design

4.2.4 Emergent Behaviors

4.3 Online Adaptation with the ESN Module

4.3.1 Capture Performance on Unseen Evaders

4.3.2 Adaptation After a Strategy Switch

4.3.3 Ablation of Reservoir Parameters

4.3.4 Computational Cost

The per-environment RLS state requires $(N_x + 1)^2$ floating-point values, which for $N_x = 500$ is about 2.5×10^5 values per environment.

4.4 Discussion

4.4.1 Limitations

Chapter 5. Conclusion

5.1 Summary

This thesis addressed the problem of learning a drone pursuit policy that remains effective against evaders that actively react to it and whose behavior may be unknown at deployment. We proposed a population-based self-play framework for one-versus-one quadrotor pursuit–evasion with CTBR control, in which each generation trains one side against frozen opponents sampled by a multi-metric, temperature-scaled hardness score. A reward design with partial credit for opponent-induced crashes and penalties on commanded body rates prevented degenerate strategies and aggressive control unsuitable for real hardware. On top of this framework, we equipped the pursuer with an echo state network whose linear readout is updated online by recursive least squares to predict the evader's relative motion, providing a lightweight, gradient-free mechanism for test-time adaptation.

5.2 Future Work

Several directions remain open. First, the ESN-equipped pursuer should be validated in real flight against both scripted and learned evaders, following the zero-shot transfer demonstrated by recent self-play systems [28, 29, 24]. Second, the framework can be extended to multiple pursuers or evaders, where cooperation among pursuers and the prediction of several opponents become important [19, 30]. Third, the observation could be replaced by onboard perception, introducing estimation noise and limited field of view as in practical interception systems [5]. Finally, the online adaptation module could be combined with adaptation to the pursuer's own dynamics [33, 34], so that a single policy adapts to both its opponent and its environment.

Bibliography

[1] D. Chauhan, H. Kagathara, H. Mewada, S. Patel, S. Kavaiya, and G. Barb, "Nation's Defense: A Comprehensive Review of Anti-Drone Systems and Strategies," *IEEE Access*, vol. 13, pp. 53 476–53 505, 2025.

[2] A. Palmas, "Reinforcement Learning for Decision-Level Interception Prioritization in Drone Swarm Defense," Sep. 2025.

[3] N. Lucotte, A. Perrusquía, A. Tsourdos, W. Guo, and H.-S. Shin, "Interception of adversarial drones in urban spaces using a modified deep reinforcement learning approach," *Applied Soft Computing*, vol. 202, p. 115797, Oct. 2026.

[4] M. Vrba, Y. Stasinchuk, T. Báča, V. Spurný, M. Petrlík, D. Heřt, D. Žaitlík, and M. Saska, "Autonomous capture of agile flying objects using UAVs: The *MBZIRC 2020* challenge," *Robotics and Autonomous Systems*, vol. 149, p. 103970, Mar. 2022.

[5] M. Pliska, M. Vrba, T. Báča, and M. Saska, "Towards Safe Mid-Air Drone Interception: Strategies for Tracking & Capture," *IEEE Robotics and Automation Letters*, vol. 9, no. 10, pp. 8810–8817, Oct. 2024.

[6] E. Kaufmann, A. Loquercio, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Deep Drone Acrobatics (Extended Abstract)," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*. Montreal, Canada: International Joint Conferences on Artificial Intelligence Organization, Aug. 2021, pp. 4780–4783.

[7] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982–987, Aug. 2023.

[8] S. A. Bahnam, R. Ferede, T. M. Blaha, A. E. Lang, E. Lucassen, Q. Missinne, A. E. C. Verraest, C. De Wagter, and G. C. H. E. de Croon, "MonoRace: Winning Champion-Level Drone Racing with Robust Monocular AI," 2026.

[9] M. Bosello, F. Pinzarrone, S. Kiade, D. Aguiari, Y. Keuter, A. AlShehhi, G. Caminati, K. L. Wong, K. S. Chou, J. Halepota, F. Alneyadi, J. Panerati, and G. Pau, "On Your Own: Pro-Level Autonomous Drone Racing in Uninstrumented Arenas," *IEEE Robotics and Automation Letters*, vol. 11, no. 3, pp. 2674–2681, Mar. 2026.

[10] E. Kaufmann, L. Bauersfeld, and D. Scaramuzza, "A Benchmark Comparison of Learned Control Policies for Agile Quadrotor Flight," in *2022 International Conference on Robotics and Automation (ICRA)*, May 2022, pp. 10 504–10 510.

[11] J. Chen, C. Yu, Y. Xie, F. Gao, Y. Chen, S. Yu, W. Tang, S. Ji, M. Mu, Y. Wu, H. Yang, and Y. Wang, "What Matters in Learning a Zero-Shot Sim-to-Real RL Policy for Quadrotor Control? A Comprehensive Study," *IEEE Robotics and Automation Letters*, vol. 10, no. 7, pp. 7134–7141, Jul. 2025.

[12] B. Xu, F. Gao, C. Yu, R. Zhang, Y. Wu, and Y. Wang, "OmniDrones: An Efficient and Flexible Platform for Reinforcement Learning in Drone Control," Sep. 2023.

[13] Z. Hu, L. Xiao, J. Guan, W. Yi, and H. Yin, "Intercept Guidance of Maneuvering Targets with Deep Reinforcement Learning," *International Journal of Aerospace Engineering*, vol. 2023, no. 1, p. 7924190, 2023.

[14] F. Giral, I. Gómez, and S. Le Clainche, "Intercepting Unauthorized Aerial Robots in Controlled Airspace Using Reinforcement Learning," 2024.

[15] A. A. Darwish and A. Nakhmani, "Drone Navigation and Target Interception Using Deep Reinforcement Learning: A Cascade Reward Approach," *IEEE Journal of Indoor and Seamless Positioning and Navigation*, vol. 1, pp. 130–140, 2023.

[16] B. Xia, I. Mantegh, and W. Xie, "UAV Multi-Dynamic Target Interception: A Hybrid Intelligent Method Using Deep Reinforcement Learning and Fuzzy Logic," *Drones*, vol. 8, no. 6, p. 226, Jun. 2024.

[17] D. Fan, H. Shen, and L. Dong, "Switching-aware multi-agent deep reinforcement learning for target interception," *Applied Intelligence*, vol. 53, no. 7, pp. 7876–7891, Apr. 2023.

[18] R. Zhang, Q. Zong, X. Zhang, L. Dou, and B. Tian, "Game of Drones: Multi-UAV Pursuit-Evasion Game With Online Motion Planning by Deep Reinforcement Learning," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 10, pp. 7900–7909, Oct. 2023.

[19] J. Chen, C. Yu, G. Li, W. Tang, S. Ji, X. Yang, B. Xu, H. Yang, and Y. Wang, "Online Planning for Multi-UAV Pursuit-Evasion in Unknown Environments Using Deep Reinforcement Learning," *IEEE Robotics and Automation Letters*, vol. 10, no. 8, pp. 8196–8203, Aug. 2025.

[20] K. Zhang, Z. Yang, and T. Başar, "Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms," in *Handbook of Reinforcement Learning and Control*, K. G. Vamvoudakis, Y. Wan, F. L. Lewis, and D. Cansever, Eds. Cham: Springer International Publishing, 2021, pp. 321–384.

[21] J. Liang, H. Miao, K. Li, J. Tan, X. Wang, R. Luo, and Y. Jiang, "A Review of Multi-Agent Reinforcement Learning Algorithms," *Electronics*, vol. 14, no. 4, p. 820, Jan. 2025.

[22] R. Lowe, YI. WU, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch, "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017.

[23] W. Li, Z. Ding, S. Karten, and C. Jin, "FightLadder: A Benchmark for Competitive Multi-Agent Reinforcement Learning," Jun. 2024.

[24] A. S. Roncero, Y. Cai, O. Andersson, and P. Ogren, "Learned Controllers for Agile Quadrotors in Pursuit-Evasion Games," Jun. 2026.

[25] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi, R. Powell, T. Ewalds, P. Georgiev, J. Oh, D. Horgan, M. Kroiss, I. Danihelka, A. Huang, L. Sifre, T. Cai, J. P. Agapiou, M. Jaderberg, A. S. Vezhnevets, R. Leblond, T. Pohlen, V. Dalibard, D. Budden, Y. Sulsky, J. Molloy, T. L. Paine, C. Gulcehre, Z. Wang, T. Pfaff, Y. Wu, R. Ring, D. Yogatama,

D. Wünsch, K. McKinney, O. Smith, T. Schaul, T. Lillicrap, K. Kavukcuoglu, D. Hassabis, C. Apps, and D. Silver, "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, Nov. 2019.

[26] C. Chen, T. Song, L. Mo, M. Lv, and D. Lin, "Autonomous Dogfight Decision-Making for Air Combat Based on Reinforcement Learning with Automatic Opponent Sampling," *Aerospace*, vol. 12, no. 3, p. 265, Mar. 2025.

[27] J. Xiao and M. Feroskhan, "Learning Multipursuit Evasion for Safe Targeted Navigation of Drones," *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 12, pp. 6210–6224, Dec. 2024.

[28] W. Tang, T. Chen, Z. Cui, B. An, J. Chen, R. Zhang, H. Liu, T. Wu, Q. Liao, F. Gao, Y. Wang, and C. Yu, "AgilePE: Autonomous UAV Pursuit-Evasion via Self-Play Reinforcement Learning," Aug. 2026.

[29] T. Gavin, S. Lacroix, and M. Bronz, "Agile Interception of a Flying Target using Competitive Reinforcement Learning," Mar. 2026.

[30] T. Gavin and M. Bronz, "Intercepting an Agile Target with Net-Carrying Drones Using Competitive Multi-Agent Reinforcement Learning," in *2026 International Conference on Unmanned Aircraft Systems (ICUAS)*, Jun. 2026, pp. 759–766.

[31] I. Geles, L. Bauersfeld, M. Wulfmeier, and D. Scaramuzza, "Superhuman Safe and Agile Racing through Multi-Agent Reinforcement Learning," 2026.

[32] Y. Tian, K.-R. Kladny, Q. Wang, Z. Huang, and O. Fink, "Multi-agent actor-critic with time dynamical opponent model," *Neurocomputing*, vol. 517, pp. 165–172, Jan. 2023.

[33] J. Pan, J. Xing, R. Reiter, Y. Zhai, E. Aljalbout, and D. Scaramuzza, "Learning on the Fly: Rapid Policy Adaptation via Differentiable Simulation," Jan. 2026.

[34] J. Xing, Z. Zhu, Y. Ren, I. Geles, Y. Zhai, R. Reiter, and D. Scaramuzza, "Continual Robot Policy Learning via Variational Neural Dynamics," Jun. 2026.

[35] G. Pertin and D. E. Chang, "NST-RL: Residual Reinforcement Learning With Approximate Optimal Regulation for Robust Drone Control," *IEEE Robotics and Automation Letters*, vol. 11, no. 10, pp. 12 200–12 207, Oct. 2026.

[36] J. Green, J. Xing, N. Messikommer, A. Romero, and D. Scaramuzza, "Bridging Performance and Generalization in Reinforcement Learning for Agile Flight," Jun. 2026.

[37] A. Yoshimura and G. Tanaka, "Online Adaptive Reinforcement Learning with Echo State Networks for Non-Stationary Dynamics," Feb. 2026.

[38] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games."

[39] B. Zhou, F. Gao, L. Wang, C. Liu, and S. Shen, "Robust and Efficient Quadrotor Trajectory Generation for Fast Autonomous Flight," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3529–3536, Oct. 2019.

[40] Z. Xu, X. Han, H. Shen, H. Jin, and K. Shimada, "NavRL: Learning Safe Flight in Dynamic Environments," *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3668–3675, Apr. 2025.

[41] H. I. Ugurlu, X. H. Pham, and E. Kayacan, "Sim-to-Real Deep Reinforcement Learning for Safe End-to-End Planning of Aerial Robots," *Robotics*, vol. 11, no. 5, p. 109, Oct. 2022.

[42] Y. Zhang, Y. Hu, Y. Song, D. Zou, and W. Lin, "Learning vision-based agile flight via differentiable physics," *Nature Machine Intelligence*, vol. 7, no. 6, pp. 954–966, Jun. 2025.

[43] S. Gurumurthy, J. Z. Kolter, and Z. Manchester, "Deep Off-Policy Iterative Learning Control," in *Proceedings of The 5th Annual Learning for Dynamics and Control Conference*. PMLR, Jun. 2023, pp. 639–652.

[44] X. Wang, H. Zhang, J. Xu, S. Wang, and M. Guay, "Reinforcement learning in pursuit-evasion differential game: Safety, stability and robustness," Jul. 2025.

[45] Z. Feng, R. Xue, L. Yuan, Y. Yu, N. Ding, M. Liu, B. Gao, J. Sun, X. Zheng, and G. Wang, "Multi-agent embodied AI: Advances and future directions," *Science China Information Sciences*, vol. 69, no. 5, p. 151202, Mar. 2026.

[46] W. Kong, D. Zhou, Z. Yang, Y. Zhao, and K. Zhang, "UAV Autonomous Aerial Combat Maneuver Strategy Generation with Observation Error Based on State-Adversarial Deep Deterministic Policy Gradient and Inverse Reinforcement Learning," *Electronics*, vol. 9, no. 7, p. 1121, Jul. 2020.

[47] Z. Xu, S. Shao, and Z. Han, "A DRL Framework for Autonomous Pursuit-Evasion: From Multi-Spacecraft to Multi-Drone Scenarios," *Drones*, vol. 9, no. 9, p. 636, Sep. 2025.

Acknowledgments in Korean

언제나 저를 바른 길로 이끌어 주시는 송익호 교수님께 큰 고마움을 느낍니다. 끝으로 오늘의 제가 있을 수 있도록 사랑으로 키워 주신 가족들에게 감사드립니다. 저의 이 작은 결실이 그분들께 조금이나마 보답이 되기를 바랍니다.

Curriculum Vitae in Korean

이　　름: 박 상 우

생 년 월 일: 19xx년 xx월 xx일

출　생　지: ...

주　　소: ...

학　　력

2008. 3. – 2010. 2.　고등학교 (2년 수료)

2010. 2. – 2014. 2.　한국과학기술원 물리학과 (학사)

2014. 3. – 2016. 2.　한국과학기술원 전기및전자공학부 (석사)

경　　력

2015. 3. – 2016. 2.　한국과학기술원 전기및전자공학부 조교

연 구 업 적

1. H.-K. Min, Y. Hou, **S. Park**, and I. Song, "A computationally efficient scheme for feature extraction with kernel discriminant analysis," *Patt. Recogn.*, vol. 50, no. 2, pp. 45-55, Feb. 2016 (to be published).